# เอกสารส่งมอบงานและคู่มือการปฏิบัติการ

## โครงการ: ระบบตรวจจับและบันทึกป้ายทะเบียนยานพาหนะ

ส่วนงาน: Backend API และระบบเซิร์ฟเวอร์ (Backend API & Server Infrastructure)

| | |
|---|---|
| **ชื่อโครงการ / ระบบ:** | ระบบตรวจจับและรู้จำป้ายทะเบียนยานพาหนะ (LPR System) |
| **ขอบเขตงานที่ส่งมอบ:** | ระบบหลังบ้าน (Backend API), ฐานข้อมูล, ระบบสตรีมภาพกล้อง (MediaMTX) และการติดตั้งบนเครื่องเซิร์ฟเวอร์ |
| **เวอร์ชันที่ส่งมอบ:** | 1.0.2 (Git tag `v1.0.2`, commit 9a7ef50) |
| **ผู้ส่งมอบงาน:** | ภูมิยศ ลาวัณย์วิสุทธิ์ ตำแหน่ง Backend Developer<br>(นักศึกษาฝึกงาน) |
| **ช่องทางติดต่อผู้ส่งมอบ:** | อีเมล: phumyod.lawanvisut@gmail.com |
| **วันที่ส่งมอบงาน:** | 9 ตุลาคม 2569 |

## 1. ข้อมูลทั่วไปและขอบเขตความรับผิดชอบเฉพาะส่วนงาน (Scope of Deliverables)

เอกสารฉบับนี้จัดทำขึ้นเพื่อส่งมอบงานพัฒนาระบบเฉพาะในส่วนงาน Backend
ซึ่งเป็นศูนย์กลางของระบบ ทำหน้าที่รับผลการตรวจจับจาก AI Vision บันทึกข้อมูล ตรวจสอบสิทธิ์ผู้ใช้ และให้บริการข้อมูลแก่หน้าเว็บ โดยมีรายละเอียดขอบเขตดังนี้:

**ขอบเขตงานที่รับผิดชอบ (In Scope):**

- พัฒนา REST API ด้วย FastAPI (Python 3.11) จำนวน 71 endpoint
- ระบบยืนยันตัวตน กำหนดสิทธิ์ 3 ระดับ และแยกข้อมูลตามหมู่บ้าน (Multi-tenant)
- รับผลการตรวจจับจาก AI Vision ผ่าน Webhook และจัดเก็บรูปภาพหลักฐาน
- ตรวจสอบบัญชีดำ / บัญชีขาว และแจ้งเตือนแบบเรียลไทม์ (SSE) ในระบบ และทางอีเมล
- จัดการกล้อง เชื่อมต่อกับ MediaMTX และ AI Vision และตรวจสอบสถานะกล้องอัตโนมัติ
- ตรวจสอบสิทธิ์การดูภาพสด (HLS) ทุกครั้งที่มีผู้ชม
- ออกแบบฐานข้อมูล PostgreSQL และจัดการ Migration ด้วย Alembic
- ติดตั้งระบบด้วย Docker Compose และ Deploy อัตโนมัติด้วย GitHub Actions
- จัดทำเอกสารประกอบระบบ: คู่มือติดตั้งและดูแลระบบ เอกสารสถาปัตยกรรม และเอกสาร API

**ขอบเขตงานที่ไม่เกี่ยวข้อง (Out of Scope):**

- หน้าเว็บแอปพลิเคชัน (Frontend Web Dashboard)
- โมเดลปัญญาประดิษฐ์ตรวจจับป้ายทะเบียน (AI/LPR Model Engine)
- ฮาร์ดแวร์กล้องวงจรปิด (CCTV Hardware)
- อุปกรณ์เครือข่าย, Proxy HTTPS ด้านหน้า และการจัดหาเครื่องเซิร์ฟเวอร์ (VM)

> **ข้อจำกัดสำคัญด้านการติดตั้ง**
>
> Backend (service `api`) ต้องรันเป็น process เดียวใน container เดียวเท่านั้น
> เนื่องจากข้อมูลเซสชัน การจำกัดจำนวนคำขอ การล็อกบัญชี
> และการแจ้งเตือนแบบเรียลไทม์เก็บอยู่ในหน่วยความจำ ห้ามเพิ่มจำนวน worker หรือจำนวน container ของ api

## 2. รายละเอียดสิ่งที่พัฒนาและส่งมอบ (Deliverables & Features)

ผู้ส่งมอบได้ดำเนินการออกแบบและพัฒนาระบบหลังบ้าน
โดยประกอบด้วยโมดูลและฟังก์ชันการทำงานหลักจำนวน 12 โมดูล ดังตารางต่อไปนี้:

| ลำดับ | โมดูล / ระบบงาน | รายละเอียดคุณสมบัติที่พัฒนา |
|---|---|---|
| 1 | ระบบยืนยันตัวตนและความปลอดภัย (Authentication & Security) | • Access Token (JWT) อายุสั้น คู่กับ Refresh Token ในคุกกี้ HTTP-Only<br>• หมุนเวียน Refresh Token ทุกครั้ง และตรวจจับการนำ Token เก่ามาใช้ซ้ำ<br>• จำกัดไม่เกิน 5 เซสชันต่อผู้ใช้ และเปลี่ยนรหัสผ่านแล้วออกจากระบบทุกเครื่อง<br>• ลืมรหัสผ่าน / ตั้งรหัสผ่าน / ยืนยันอีเมลใหม่ ผ่านลิงก์ทางอีเมล<br>• เข้ารหัสรหัสผ่านด้วย Argon2 พร้อม Password Policy<br>• ล็อกบัญชีเมื่อใส่รหัสผิดซ้ำ และแจ้งเตือนผู้ดูแลเมื่อพบการเดารหัสผ่าน<br>• จำกัดจำนวนคำขอ (Rate Limit) ทั้งระบบและรายendpoint |

| ลำดับ | โมดูล / ระบบงาน | รายละเอียดคุณสมบัติที่พัฒนา |
|---|---|---|
| 2 | ระบบผู้ใช้และสิทธิ์การเข้าถึง (Users & Role-Based Access Control) | • สิทธิ์ 3 ระดับ: superadmin, admin และ user<br>• แยกข้อมูลตามหมู่บ้าน ผู้ใช้เห็นเฉพาะข้อมูลหมู่บ้านของตน<br>• ส่งคำเชิญทางอีเมล, รีเซ็ตรหัสผ่าน, ปลดล็อกบัญชี และรูปโปรไฟล์ |
| 3 | ระบบจัดการหมู่บ้าน (Village / Multi-tenancy) | • เพิ่ม / แก้ไข / ปิดใช้งาน / ลบหมู่บ้าน (เฉพาะ superadmin)<br>• ปิดหมู่บ้านแล้วปิดกล้องทั้งหมดของหมู่บ้านอัตโนมัติ<br>• ลบหมู่บ้านแล้วยังเก็บประวัติการตรวจจับไว้เป็นสถิติ |
| 4 | ระบบจัดการกล้องวงจรปิด (Camera Management) | • เพิ่ม / แก้ไข / ลบกล้อง พร้อมลงทะเบียนกับ MediaMTX และ AI Vision อัตโนมัติ<br>• ยืนยันว่า AI Vision อ่านภาพจากกล้องได้ (pending ⟶ verified / failed)<br>• ตรวจสถานะออนไลน์ของกล้องทุก 10 วินาที และแจ้งเตือนเมื่อกล้องหลุด<br>• ค้นหา RTSP URL จากกล้องที่รองรับ ONVIF |
| 5 | ระบบรับผลการตรวจจับ (Detection Ingestion) | • Webhook รับข้อมูลแบบ multipart พร้อมรูปป้ายทะเบียนและรูปเต็มของรถ<br>• ป้องกันการบันทึกซ้ำด้วย event_id และมีโหมดทดสอบการเชื่อมต่อ<br>• ตรวจสอบไฟล์รูป (JPEG / PNG) และจัดเก็บแยกตามหมู่บ้านและกล้อง |
| 6 | ระบบบัญชีดำ บัญชีขาว และการเตือนภัย (Blacklist, Whitelist & Alerts) | • จัดการรายการป้ายทะเบียนแยกตามหมู่บ้าน<br>• ตรวจพบป้ายใน Blacklist แล้วแจ้งเตือนแบบเรียลไทม์ในระบบ และทางอีเมล (ป้ายเดิมไม่ส่งอีเมลซ้ำภายใน 15 นาที) |
| 7 | ระบบภาพสดจากกล้อง (Live Streaming) | • MediaMTX ดึงภาพ RTSP จากกล้องและแปลงเป็น HLS<br>• ออก Token ดูภาพ (ES256) อายุสั้น และตรวจสิทธิ์ผู้ชมทุกครั้ง |

| ลำดับ | โมดูล / ระบบงาน | รายละเอียดคุณสมบัติที่พัฒนา |
|---|---|---|
| | | • ปฏิเสธการส่งภาพเข้าเซิร์ฟเวอร์จากภายนอก |
| 8 | ระบบข้อมูลเรียลไทม์ (Server-Sent Events) | • ช่องแจ้งเตือนทั่วไป ช่องแจ้งเตือนความปลอดภัย และสถานะผู้ใช้ที่ออนไลน์<br>• เชื่อมต่อด้วย Ticket ที่ใช้ได้ครั้งเดียว และตัดการเชื่อมต่อเมื่อเซสชันถูกยกเลิก |
| 9 | ระบบสืบค้น รายงาน และติดตามเส้นทาง (Search, Reports & Route Tracking) | • ค้นหาประวัติตามป้ายทะเบียน (ค้นหาบางส่วนได้) จังหวัด กล้อง และช่วงเวลา<br>• แดชบอร์ดสรุปวันนี้ รายงานสรุป และรายงานรายวัน<br>• ติดตามเส้นทางของรถผ่านกล้องหลายตัว |
| 10 | ระบบบันทึกประวัติและการแจ้งเตือน (Audit Logs & Notifications) | • บันทึกการกระทำสำคัญพร้อม IP และ User-Agent<br>• การแจ้งเตือนในระบบรายผู้ใช้ และช่องทางติดต่อของผู้ใช้ |
| 11 | ฐานข้อมูล (Database) | • PostgreSQL 15 จำนวน 11 ตาราง พร้อม Index สำหรับการค้นหา<br>• จัดการโครงสร้างฐานข้อมูลด้วย Alembic Migration |
| 12 | การติดตั้งผ่าน Docker และ Deploy (Containerization & CI/CD) | • Docker Compose 4 บริการ: nginx, api, db และ mediamtx<br>• Image ของ api รันด้วยผู้ใช้ที่ไม่ใช่ root<br>• Deploy อัตโนมัติเมื่อ push เข้า branch main ผ่าน GitHub Actions (Self-hosted Runner) |

## 3. แหล่งจัดเก็บซอร์สโค้ดและสภาพแวดล้อมระบบ (Code & Storage Location)

ซอร์สโค้ดและไฟล์ประกอบระบบทั้งหมดถูกจัดเก็บและควบคุมเวอร์ชันตามรายละเอียดดังนี้:

| | |
|---|---|
| **Git Remote Repository:** | https://github.com/dev-planetcloud/lpr-backend.git |
| **Primary Branch / Tag:** | `main` / `v1.0.2` |

| | |
|---|---|
| **เครื่องเซิร์ฟเวอร์ (Production):** | VM 192.168.100.97 โฟลเดอร์ /home/trainee/YOLO |
| **URL ใช้งาน:** | https://trainee.api.planetcloud.cloud |
| **Production Services:** | village_guard_nginx (พอร์ต 80), village_guard_api, village_guard_db, village_guard_mediamtx |
| **ตำแหน่งข้อมูล:** | ฐานข้อมูลใน Docker volume pgdata, รูปภาพในโฟลเดอร์ storage/, ไฟล์สำรองที่ ~/backups บนเครื่อง VM เดียวกัน ต้องคัดลอกสำเนาไปเก็บนอกเครื่องเป็นประจำตามหัวข้อ 4.5 ขั้นตอนที่ 4 |

**โครงสร้างและไฟล์สำคัญในระบบ:**

| ไฟล์ / โฟลเดอร์ | คำอธิบาย |
|---|---|
| app/ | ซอร์สโค้ด Python (FastAPI) แบ่งเป็น api (endpoint), core (ความปลอดภัยและค่าตั้ง), models (ตารางฐานข้อมูล), schemas (รูปแบบข้อมูล) และ services (การทำงานหลัก) |
| alembic/ | ไฟล์ Migration ของฐานข้อมูล |
| docker-compose.yml | กำหนดบริการทั้ง 4 ตัว พอร์ต และ volume |
| dockerfile | สคริปต์สร้าง image ของ api (Python 3.11) |
| nginx/nginx.conf | Reverse proxy แยกเส้นทาง /, /api/, /mediamtx/ และ /smartlpr/ |
| mediamtx.yml | ค่าตั้งของระบบสตรีมภาพกล้อง |
| .github/workflows/deploy.yml | ขั้นตอน Deploy อัตโนมัติ |
| create_superadmin.py | สคริปต์สร้างผู้ดูแลระบบสูงสุด (superadmin) คนแรก |
| README.md | คู่มือติดตั้ง อัปเดต ดูแลระบบ และแก้ปัญหาเบื้องต้น |
| docs/ARCHITECTURE.md | สถาปัตยกรรม ฐานข้อมูล สิทธิ์ผู้ใช้ และ flow การทำงาน |
| .env.example | ตัวอย่างและคำอธิบายตัวแปรสภาพแวดล้อมทุกตัว |

**ตัวอย่างการตั้งค่าในไฟล์ .env (อ้างอิงจาก .env.example ซึ่งอธิบายครบทุกตัว):**

| ตัวแปร | ค่า / คำอธิบาย |
|---|---|
| `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` | ข้อมูลฐานข้อมูล |
| `JWT_SECRET` | สร้างใหม่ด้วยคำสั่ง `openssl rand -hex 32` |
| `API_KEY` | key ที่ AI Vision ใช้ส่งผลการตรวจจับเข้ามา |
| `AI_VISION_API_URL` | `http://<IP_AI_VISION>:8081` |
| `MEDIAMTX_API_URL` | `http://mediamtx:9997` (ห้ามใช้ IP ของเครื่อง) |
| `MEDIAMTX_JWT_PRIVATE_KEY_B64` | EC private key แบบ base64 สำหรับ Token ดูภาพสด |
| `SMTP_HOST`, `SMTP_USER`, `SMTP_PASSWORD` | บัญชีสำหรับส่งอีเมล |
| `FRONTEND_URL`, `BACKEND_PUBLIC_URL` | https://[โดเมน] |

**พอร์ตของระบบ:**

| พอร์ต | บริการ | เข้าถึงได้จาก |
|---|---|---|
| 80 | nginx (หน้าเว็บ, API, ภาพสด) | ทุกเครื่อง (ผ่าน Proxy HTTPS ด้านหน้า) |
| 8554 / 8888 | MediaMTX (RTSP / HLS) | เครือข่ายภายใน |
| 5432 | PostgreSQL | เฉพาะบนเครื่อง VM |
| 9997 | MediaMTX API | เฉพาะบนเครื่อง VM |

## 4. คู่มือการรับมือเมื่อระบบขัดข้องและการกู้คืนระบบ (Troubleshooting & Disaster Recovery)

ในกรณีที่เกิดปัญหา ให้เข้า Terminal ของเครื่องเซิร์ฟเวอร์ ไปที่โฟลเดอร์ `/home/trainee/YOLO` แล้วตรวจสอบและแก้ไขตามลำดับอาการดังต่อไปนี้ (รายละเอียดเพิ่มเติมอยู่ใน README.md หัวข้อ 6):

### 4.1 กรณีที่ 1: เรียก API ไม่ได้ หรือหน้าเว็บแจ้ง 502 (Backend Container Down)

**อาการ:** หน้าเว็บเปิดได้แต่แจ้งเชื่อมต่อเซิร์ฟเวอร์ไม่ได้ หรือ /api ตอบ 502 Bad Gateway

**ขั้นตอนที่ 1:** ตรวจสอบสถานะคอนเทนเนอร์:

```
docker compose ps
```

**ขั้นตอนที่ 2:** ดูข้อความแจ้งความผิดพลาดของ api:

```
docker compose logs --tail 100 api
```

**ขั้นตอนที่ 3:** ถ้า log มีคำว่า `Field required` แปลว่าไฟล์ .env ขาดตัวแปร ให้เทียบกับ .env.example แล้วสั่ง:

```
docker compose up -d
```

**ขั้นตอนที่ 4:** สั่งเริ่มการทำงานใหม่ (ถ้ามีการแก้ .env ต้องใช้ `docker compose up -d` แทน เพราะ restart ไม่อ่าน .env ใหม่):

```
docker compose restart api
```

**ขั้นตอนที่ 5:** ตรวจว่า api กลับมาทำงาน ต้องได้ผลลัพธ์ `{"status":"ok"}`:

```
docker compose exec api python -c "import urllib.request;
print(urllib.request.urlopen('http://localhost:8000/health').read().decode())"
```

### 4.2 กรณีที่ 2: ภาพสดไม่แสดง หรือกล้องขึ้นออฟไลน์ทุกตัว (MediaMTX)

**อาการ:** หน้า Live Monitor แสดงจอดำทุกกล้อง หรือได้รับแจ้งเตือน streaming_server_down

**ขั้นตอนที่ 1:** ตรวจสถานะและ log ของ MediaMTX:

```
docker compose ps mediamtx
docker compose logs --tail 100 mediamtx
```

**ขั้นตอนที่ 2:** ตรวจไฟล์ .env ว่า `MEDIAMTX_API_URL=http://mediamtx:9997`

**ขั้นตอนที่ 3:** ดูรายการกล้องใน MediaMTX (รันบน VM) ทุกกล้องต้องมี `"ready": true`:

```
curl http://127.0.0.1:9997/v3/paths/list
```

**ขั้นตอนที่ 4:** ถ้ากล้องหายจากรายการ สั่ง `POST /api/cameras/resync-all` ด้วยบัญชี admin

**ขั้นตอนที่ 5:** ถ้าเป็นเฉพาะบางกล้อง ให้ตรวจว่ากล้องจริงออนไลน์ จ่ายไฟและเครือข่ายปกติ และ RTSP URL ถูกต้อง

### 4.3 กรณีที่ 3: ผลการตรวจจับไม่เข้าระบบ (AI Vision Webhook)

**อาการ:** กล้องออนไลน์ แต่ไม่มีรายการตรวจจับใหม่ในหน้าประวัติ

**ขั้นตอนที่ 1:** ดูคำขอที่ AI Vision ส่งเข้ามาใน 10 นาทีล่าสุด:

```
docker compose logs --since 10m api | grep detections
```

**ขั้นตอนที่ 2:** ดูรหัสผลลัพธ์ที่ AI Vision ได้รับ:

| รหัส | ความหมาย |
|---|---|
| 401 | API key ไม่ตรงกับค่า `API_KEY` ใน .env |
| 404 | camera_id ไม่มีอยู่ในระบบ |
| 409 | กล้องหรือหมู่บ้านถูกปิดใช้งาน |
| 422 | รูปแบบข้อมูลไม่ถูกต้อง เช่น capture_time ต้องเป็น YYYY/MM/DD HH:MM:SS |
| 429 | ส่งถี่เกิน 200 ครั้งต่อนาทีต่อกล้อง |

**ขั้นตอนที่ 3:** ให้ทีม AI Vision ทดสอบการเชื่อมต่อด้วย event_id ที่ขึ้นต้นด้วย `TEST_Event_` ระบบต้องตอบ 200 โดยไม่บันทึกข้อมูล

**ขั้นตอนที่ 4:** ตรวจสถานะการยืนยันกล้องในหน้าจัดการกล้อง ถ้าเป็น failed ให้สั่งตรวจซ้ำด้วย `POST /api/cameras/{id}/verification-check`

### 4.4 กรณีที่ 4: เข้าสู่ระบบไม่ได้หรือถูกปฏิเสธคำขอ (Login & Rate Limit)

| อาการ | วิธีแก้ไข |
|---|---|
| ได้รหัส 423 (บัญชีถูกล็อก) | admin ปลดล็อกในหน้าจัดการผู้ใช้ หรือรอให้หมดเวลาล็อก |
| ผู้ใช้ทุกคนได้รหัส 429 พร้อมกัน | ค่า `TRUST_PROXY_HEADERS` / `TRUSTED_PROXY_HOPS` ไม่ตรงกับจำนวน proxy ด้านหน้า ให้แก้ตาม .env.example |
| เข้าสู่ระบบได้ แต่ถูกออกจากระบบหลังประมาณ 15 นาที | เปิดเว็บผ่าน http ขณะที่ `COOKIE_SECURE=true` ให้เข้าผ่าน https เท่านั้น |

| อาการ | วิธีแก้ไข |
|---|---|
| อีเมลลืมรหัสผ่าน / คำเชิญไม่ถูกส่ง | ตรวจค่า SMTP_* ใน .env และดู log ของ api (Gmail ต้องใช้ App Password) |

## 5. เอกสารแนบ (Attachments)

| # | เอกสาร | ตำแหน่ง |
|---|---|---|
| 1 | คู่มือติดตั้ง อัปเดต ดูแลระบบ และแก้ปัญหาเบื้องต้น | `README.md` |
| 2 | เอกสารสถาปัตยกรรม ฐานข้อมูล สิทธิ์ผู้ใช้ และ flow การทำงาน | `docs/ARCHITECTURE.md` |
| 3 | คำอธิบายตัวแปรสภาพแวดล้อม | `.env.example` |
| 4 | เอกสาร API (Swagger) และฉบับ PDF | `/docs` และ API_Document_Planetcomm_LPR.pdf (แนบในหน้า Releases ของ repository) |